Block Witness

개요

Block Witness는 ZKEVM에서 블록 단위의 증명을 생성하기 위해 필요한 모든 데이터를 담고 있는 핵심 구조체입니다. `Block` 구조체는 트랜잭션, 서명, 실행 단계, RW 연산, 바이트코드 등 블록 실행과 관련된 모든 정보를 포함하며, 이를 통해 각 회로가 필요로 하는 witness 데이터를 생성합니다.

Block 구조체

정의

`Block` 구조체는 `zkevm-circuits/src/witness/block.rs` 에 정의되어 있으며, 다음과 같은 주요 필드를 포함합니다:

```
pub struct Block {
/// 블록에 포함된 트랜잭션들
pub txs: Vec<Transaction>,
/// 블록의 서명들
pub sigs: Vec<Signature>,
/// 마지막 트랜잭션 이후 반복되는 패딩 단계
pub padding_step: ExecStep,
/// EVM의 마지막 행에 나타나는 EndBlock 단계
pub end_block_step: ExecStep,
/// RwTable의 Read-Write 이벤트들
```

```rust
pub rws: RwMap,
/// 블록에서 사용된 바이트코드
pub bytecodes: BTreeMap<Word, Bytecode>,
/// 바이트코드 맵 (dual-bytecode 기능용)
pub bytecode_map: Option<BTreeMap<Word, bool>>,
/// 블록 컨텍스트
pub context: BlockContexts,
/// Copy Circuit의 테이블을 위한 복사 이벤트
pub copy_events: Vec<CopyEvent>,
/// Exponentiation Circuit의 테이블을 위한 지수 연산 트레이스
pub exp_events: Vec<ExpEvent>,
/// 회로 설정 파라미터
pub circuits_params: CircuitsParams,
/// SHA3 opcode의 입력들
pub sha3_inputs: Vec<Vec<u8>>,
/// 이전 블록의 상태 루트
pub prev_state_root: H256,
/// Withdraw 루트
pub withdraw_root: Word,
/// 이전 블록의 Withdraw 루트
pub prev_withdraw_root: Word,
/// MPT 업데이트
pub mpt_updates: MptUpdates,
```

```
/// 체인 ID
pub chain_id: u64,
/// StartL1QueueIndex
pub start_l1_queue_index: u64,
/// 프리컴파일 호출의 입출력
pub precompile_events: PrecompileEvents,
}
```

주요 컴포넌트

1. 트랜잭션 데이터 (txs, sigs)

- **txs**: 블록에 포함된 모든 트랜잭션의 witness 데이터
- **sigs**: 각 트랜잭션의 ECDSA 서명 (Sig Circuit에서 사용)

2. 실행 단계 (padding_step, end_block_step)

- **padding_step**: 마지막 트랜잭션 이후 EVM Circuit의 행을 채우기 위한 패딩 단계
- **end_block_step**: 블록의 끝을 표시하는 특수 단계

3. RW 연산 (rws)

- **RwMap**: 모든 Read-Write 연산을 저장하는 맵
- State Circuit에서 상태 변경을 검증하는 데 사용

4. 바이트코드 (bytecodes, bytecode_map)

- **bytecodes**: 코드 해시를 키로 하는 바이트코드 맵
- **bytecode_map**: dual-bytecode 기능이 활성화된 경우, 각 바이트코드가 첫 번째 또는 두 번째 Bytecode Circuit에 속하는지 표시

5. 블록 컨텍스트 (context)

- **BlockContexts**: 블록 번호별 블록 컨텍스트를 저장
- 각 `BlockContext` 는 coinbase, gas_limit, number, timestamp, difficulty, base_fee 등을 포함

6. 이벤트 데이터

- **copy_events**: Copy Circuit을 위한 메모리/스토리지 복사 이벤트
- **exp_events**: Exp Circuit을 위한 지수 연산 이벤트
- **sha3_inputs**: Keccak Circuit을 위한 SHA3 입력 데이터
- **precompile_events**: 프리컴파일 호출 데이터 (SHA256, MODEXP, EC 연산 등)

7. MPT 관련 (mpt_updates, prev_state_root, withdraw_root)

- **mpt_updates**: Merkle Patricia Trie 업데이트 정보
- **prev_state_root**: 이전 블록의 상태 루트
- **withdraw_root**: L2에서 L1으로의 출금 루트

블록 데이터 수집

block_convert 함수

`block_convert` 함수는 `CircuitInputBuilder` 가 생성한 블록 데이터를 `Block` witness 구조체로 변환합니다:

```
pub fn block_convert(
block: &circuit_input_builder::Blocks,
code_db: &eth_types::state_db::CodeDB,
```

```
) -> Result<Block, Error>
```

변환 과정

1. **RW 연산 수집**

```
let  rws = RwMap::from(&block.container);
rws.check_value()?;
rws.check_rw_counter_sanity();
```

- Bus-Mapping에서 생성된 RW 연산을 RwMap으로 변환
- 값의 일관성과 RW 카운터의 정합성 검증

2. **실행 단계 변환**

```
let  padding_step = step_convert(&block.block_steps.padding_step, last_block_num
let  end_block_step = step_convert(&block.block_steps.end_block_step, last_block
```

- 패딩 단계와 EndBlock 단계를 witness 형식으로 변환

3. **MPT 업데이트 생성**

```
let  mpt_updates = MptUpdates::from_unsorted_rws_with_state_roots(
&rws.table_assignments_unsorted(),
block.prev_state_root,
block.end_state_root(),
);
```

- RW 연산으로부터 MPT 업데이트 정보 추출
- 이전 상태 루트와 최종 상태 루트 연결

4. **바이트코드 수집**

```
let bytecodes: BTreeMap<Word, Bytecode> = get_bytecodes(code_db);
let bytecode_map = if cfg!(feature = "dual-bytecode") {
Some(get_bytecode_map(&bytecodes))
} else {
None
};
```

- CodeDB로부터 모든 바이트코드 추출
- dual-bytecode 기능이 활성화된 경우 바이트코드 분할

5. **트랜잭션 변환**

```
txs: block
.txs()
.iter()
.enumerate()
.map(|(idx, tx)| {
let next_block_num = if idx + 1 < num_txs {
block.txs()[idx + 1].block_num
} else {
last_block_num + 1
```

```
};
tx_convert(tx, idx + 1, chain_id, next_block_num)
})
.collect(),
```

- 각 트랜잭션을 witness 형식으로 변환
- 트랜잭션 ID와 블록 번호 할당

바이트코드 처리

get_bytecodes 함수

```
pub fn get_bytecodes(code_db: &CodeDB) -> BTreeMap<Word, Bytecode> {
code_db
.0
.iter()
.map(|(code_hash, bytes)| {
let hash = Word::from_big_endian(code_hash.as_bytes());
(
hash,
Bytecode {
hash,
bytes: bytes.clone(),
},
)
```

```
})
.collect()
}
```

- CodeDB의 모든 바이트코드를 코드 해시를 키로 하는 맵으로 변환

get_bytecode_map 함수 (dual-bytecode 기능)

```
pub fn get_bytecode_map(bytecodes: &BTreeMap<Word, Bytecode>) -> BTreeMap<Word
let bytecode_pairs = bytecodes
.iter()
.map(|(hash, bytecode)| (*hash, bytecode.bytes.len()))
.collect_vec();
let partition_result = greedy_simple_partition(bytecode_pairs.clone());
// 각 바이트코드가 첫 번째 또는 두 번째 서브 회로에 속하는지 매핑
bytecode_pairs
.iter()
.map(|(hash, len)| {
if partition_result.first_part.contains(&(*hash, *len)) {
(*hash, true)
} else {
(*hash, false)
}
})
.collect()
```

```
}
```

- 바이트코드를 두 개의 Bytecode Circuit으로 분할
- 각 바이트코드가 어느 서브 회로에 속하는지 표시

BlockContext 구조체

정의

```
pub struct BlockContext {
/// 블록의 마이너 주소
pub coinbase: Address,
/// 블록의 가스 한도
pub gas_limit: u64,
/// 블록 번호
pub number: Word,
/// 블록 타임스탬프
pub timestamp: Word,
/// 블록 난이도
pub difficulty: Word,
/// 베이스 수수료
pub base_fee: Word,
/// 이전 블록들의 해시
pub history_hashes: Vec<Word>,
```

```
/// 체인 ID
pub chain_id: u64,
/// 부모 블록 해시
pub parent_hash: H256,
/// 이 블록의 상태 루트
pub state_root: H256,
}
```

Block Table 할당

`BlockContext` 는 Block Table에 다음과 같은 데이터를 할당합니다:

```
pub fn table_assignments<F: Field>(
&self,
num_txs: usize,
cum_num_txs: usize,
num_all_txs: u64,
challenges: &Challenges<Value<F>>,
) -> Vec<[Value<F>; 3]>
```

할당되는 필드:

- **Coinbase**: 마이너 주소
- **Timestamp**: 블록 타임스탬프
- **Number**: 블록 번호

- **Difficulty**: 블록 난이도 (RLC 인코딩)
- **GasLimit**: 가스 한도
- **BaseFee**: 베이스 수수료 (RLC 인코딩)
- **ChainId**: 체인 ID
- **NumTxs**: 이 블록의 트랜잭션 수
- **CumNumTxs**: 누적 트랜잭션 수
- **NumAllTxs**: 전체 트랜잭션 수
- **BlockHash**: 이전 블록들의 해시 (각각 별도 행)

Block의 주요 메서드

1. 블록 번호 조회

```
pub fn first_block_number(&self) -> U256
pub fn last_block_number(&self) -> U256
```

- 청크에 포함된 첫 번째 및 마지막 블록 번호 반환

2. 상태 루트 조회

```
pub fn post_state_root(&self) -> H256
```

- 청크 실행 후의 상태 루트 반환
- MPT 업데이트로부터 계산된 루트와 블록 헤더의 루트를 비교

3. MPT 업데이트 적용

```
pub fn apply_mpt_updates(&mut self, mpt_state: &MptState)

pub fn apply_mpt_updates_and_update_mpt_state(&mut self, mpt_state: &mut MptS
```

- MPT 상태를 재생하여 MPT witness 생성
- 계산된 MPT 업데이트로 MPT 상태 갱신

4. 서명 데이터 추출

```
pub(crate) fn get_sign_data(

&self,

padding: bool,

) -> Vec<SignData<secp256k1::Fq, Secp256k1Affine>>
```

- 트랜잭션 서명과 ecRecover 호출의 서명 데이터 수집
- Sig Circuit에서 사용

```
pub(crate) fn get_sign_data_p256(

&self,

padding: bool,

max_p256: usize,

) -> Vec<SignData<secp256r1::Fq, Secp256r1Affine>>
```

- P256 서명 검증 데이터 수집

5. 프리컴파일 연산 추출

```
pub(crate) fn get_ec_add_ops(&self) -> Vec<EcAddOp>
pub(crate) fn get_ec_mul_ops(&self) -> Vec<EcMulOp>
pub(crate) fn get_ec_pairing_ops(&self) -> Vec<EcPairingOp>
pub(crate) fn get_big_modexp(&self) -> Vec<BigModExp>
pub(crate) fn get_sha256(&self) -> Vec<SHA256>
```

- 각 프리컴파일 회로에 필요한 연산 데이터 추출

6. 바이트코드 분할 (dual-bytecode)

```
pub(crate) fn is_first_bytecode_circuit(&self, code_hash: &U256) -> bool
pub(crate) fn get_bytecodes_for_dual_sub_circuits(&self) -> (Vec<&Bytecode>, Ve
```

- 특정 바이트코드가 첫 번째 서브 회로에 속하는지 확인
- 두 개의 Bytecode Circuit을 위한 바이트코드 분할

디버깅 및 분석 메서드

1. RW 연산 디버깅

```
pub(crate) fn debug_print_txs_steps_rw_ops(&self)
```

- 각 트랜잭션의 각 단계에서 발생한 RW 연산 출력

2. EVM Circuit 행 사용량 출력

```
pub(crate) fn print_evm_circuit_row_usage(&self)
```

- 각 ExecutionState별 행 사용량 통계 출력
- 상위 10개 ExecutionState의 사용 비율 표시

3. RW 사용량 출력

```
pub(crate) fn print_rw_usage(&self)
```

- Opcode별 메모리 및 스택 RW 연산 통계
- 각 RwTableTag별 총 연산 수 출력

더미 블록 생성

dummy_witness_block 함수

```
pub fn dummy_witness_block(chain_id: u64) -> Block
```

- 키 생성(key-gen)에 사용할 빈 witness 블록 생성
- 최소한의 구조만 포함하는 유효한 블록

관련 구조체

BlockContexts

```
pub struct BlockContexts {
```

```
/// 블록 번호를 블록 컨텍스트에 매핑
pub  ctxs: BTreeMap<u64, BlockContext>,
}
```

- 여러 블록의 컨텍스트를 관리
- 멀티 블록 청킹에서 사용

변환 구현

```
impl From<&circuit_input_builder::Blocks> for BlockContexts {
fn  from(block: &circuit_input_builder::Blocks) -> Self {
Self {
ctxs: block
.blocks
.values()
.map(|block| {
(
block.number.as_u64(),
BlockContext {
coinbase: block.coinbase,
gas_limit: block.gas_limit,
// ... 기타 필드
},
)
```

```
})
.collect::<BTreeMap<_, _>>(),
}
}
}
```

요약

Block Witness는 ZKEVM의 모든 회로가 필요로 하는 데이터를 중앙에서 관리하는 핵심 구조체입니다:

1. **데이터 통합**: 트랜잭션, RW 연산, 바이트코드, 이벤트 등 모든 실행 데이터를 하나의 구조체에 통합
2. **회로 간 공유**: 각 회로가 필요한 데이터를 Block에서 추출하여 사용
3. **변환 파이프라인**: Bus-Mapping의 출력을 회로 입력으로 변환하는 중간 단계
4. **검증 기능**: RW 연산의 일관성, 상태 루트의 정합성 등을 검증
5. **최적화 지원**: dual-bytecode, 프리컴파일 분리 등 다양한 최적화 기능 지원

Block Witness는 `block_convert` 함수를 통해 생성되며, 이후 각 회로의 witness 생성 함수에 전달되어 회로별 witness 데이터를 생성하는 데 사용됩니다.